

Basic Programming Script Language Standard

Document Version: Basic Programming Script Language Standard 2026 (Revision 2)

Abbreviation: BPS-26.2

Revision Date: July 21, 2026

Reference Implementation: Basic Programming Script 0.0.2.3 (hereinafter referred to as BPS)

1. Overview

Basic Programming Script is a lightweight interpreted scripting language designed for network automation tasks. This standard defines the syntax, semantics, and runtime behavior of the BPS language.

1.1 Design Goals

- (1) Concise syntax with low learning curve
- (2) Native HTTP client/server support
- (3) No external dependencies

1.2 Standards Conformance

This implementation conforms to all requirements of the BPS-26.2 standard.

2. Lexical Structure

2.1 Character Set

- (1) Supports ASCII character set
- (2) Unicode supported in string literals

2.2 Comments

Type	Syntax	Example
Single-line comment	!!	!!This is a comment
Multi-line comment	/! ... !/	/! Multi-line !/

2.3 Keywords

2.3.1 Variables and Flow Control

No.	Keyword	Description
1	DEFINE	Define variable
2	IF	Conditional judgment
3	ELSE	Else
4	WHILE	Loop
5	RETURN	Return value
6	FUNCTION	Function definition
7	MAIN	Main function
8	WAIT	Wait (seconds)
9	CALL	Declare access to global variable
10	TO	Array index range qualifier
11	CONST	Constant

2.3.2 Input/Output

No.	Keyword	Description
1	OUTPUT	Output
2	INPUT	Input
3	INTO	Input to variable

2.3.3 HTTP Client

No.	Keyword	Description
1	GET	GET request
2	POST	POST request
3	PUT	PUT request
4	PATCH	PATCH request
5	DELETE	DELETE request
6	WITH	Carry data
7	SAVE	Save response

2.3.4 HTTP Server

No.	Keyword	Description
1	CREATESERVER	Create server
2	ROUTE	Route
3	RESPONSE	Response
4	FILE	File response
5	STATIC	Static directory

2.3.5 Literals

No.	Keyword	Description
1	true	Boolean true
2	false	Boolean false

2.3.6 Query Expressions and Statements

No.	Keyword	Description
1	TYPE	Query value type (expression)

2.4 Operators

Type	Operator
Arithmetic	+, -, *, /, %
Comparison	==, !=, >, <, >=, <=
Assignment	=

2.5 Delimiters

() { } , ; " []

2.6 Literals

Type	Example
Integer	123, -456
Floating point	3.14, -0.5
String	"hello", "nihao"
Boolean	true, false

2.7 Escape Sequences

Sequence	Meaning
\NL	Newline
\TAB	Tab
\"	Double quote
\\	Backslash

3. Type System

3.1 Supported Types

Type	Description	Example
null	Null value	Default value for uninitialized variables
number	64-bit floating point	10, 3.14
string	String	"hello"
bool	Boolean value	true, false
array	Array, can store elements of any type	[10, 20, 30]

3.2 Type Conversion

(1) String Concatenation Conversion Rules

Left Operand Type	Right Operand Type	Result Type	Conversion Rule
string	string	string	Direct concatenation
string	number	string	Number→decimal string
string	bool	string	true→"true", false→"false"
string	null	string	null→"null"
number	string	string	Number→decimal string
number	number	number	Numeric addition
number	bool	number	bool → number: true → 1, false→0
number	null	number	null→0
bool	string	string	true→"true", false→"false"
bool	number	number	bool → number: true → 1, false→0

bool	bool	number	Both converted to numbers, then added
bool	null	number	bool→number, null→0
null	string	string	null→"null"
null	number	number	null→0
null	bool	number	null→0, bool→number
null	null	number	0 + 0 = 0

Examples:

"value:" + 100 → "value:100"

"flag:" + true → "flag:true"

"empty:" + null → "empty:null"

100 + "px" → "100px"

Note: String concatenation conversion is only performed when at least one operand is a string. Adding two numbers remains a numeric operation.

(2) Comparison Operation Conversion Rules

When using comparison operators (==, !=, >, <, >=, <=), the following conversion rules apply:

Comparison Type	Conversion Rule	Description
Same type comparison	Direct comparison	Compare according to type's own rules
Number vs String	Convert string to number	If conversion fails, compare as strings
Boolean vs Non-boolean	Convert boolean to number	true→1, false→0
Null comparison	Null only equals null	Comparing with any non-null value yields false

Specific Conversion Behavior:

① Equality Comparison ==

Number and String: String is converted to number before comparison

Boolean and Number: true→1, false→0

Boolean and String: Boolean converted to "true"/"false" before comparison

null vs null: true

null vs non-null: false

② Inequality Comparison !=

Logical opposite of ==

③ Ordering Comparison >, <, >=, <=

Both operands are numbers: numeric comparison

One number, one string: string converted to number before comparison

Both strings: lexicographic comparison

Boolean in comparison: converted to number (true=1, false=0)

Null in comparison: null converted to 0

Examples:

5 == "5" → true (string to number)

5 == 5 → true

true == 1 → true (boolean to number)

true == "true" → true (boolean to string)

null == null → true

null == 0 → false

5 > "3" → true (string to number)

"10" > "2" → false (lexicographic: "1" vs "2")

`true > 0` $\rightarrow$ `true (true $\rightarrow$ 1)`

Note: Type conversion is only performed temporarily during comparison and does not change the operand's original type.

4. Variables

4.1 Variable and Array Definitions

`DEFINE varName;` `!!` Defined as null

`DEFINE varName = expr;` `!!` Define and assign

`DEFINE varName dimDecl = arrayInit;`

4.1.1 Dimension Declaration

Syntax	Description	Example
<code>[N]</code>	Starts from 0, length N	<code>[3]</code> $\rightarrow$ indices 0, 1, 2
<code>[N TO M]</code>	From N to M	<code>[1 TO 3]</code> $\rightarrow$ indices 1, 2, 3
<code>[]</code>	Auto-deduced length (starts from 0)	<code>[]</code> $\rightarrow$ determined by initialization data
<code>[N][M]</code>	Multi-dimensional array	<code>[2][3]</code> $\rightarrow$ 2 rows, 3 columns

4.1.2 Array Initialization

`{ elem1, elem2, elem3, ... }`

Example

`!!` 1D array

`DEFINE a[3] = {10, 20, 30};`

`DEFINE a[1 TO 3] = {10, 20, 30};`

`DEFINE a[] = {10, 20, 30, 40};` `!!` Auto-deduced as `[4]`

`!!` 2D array

`DEFINE b[2][3] = {1,2,3}, {4,5,6};`

`DEFINE b[1 TO 2][0 TO 2] = {1,2,3}, {4,5,6};`

`!!` 3D array

`DEFINE c[2][2][2] = {1,2}, {3,4}, {5,6}, {7,8};`

```
!! Empty array
DEFINE empty[3] = {};
```

Rules

Rule	Description
Dimension	Determined by number of []
Element count	Init elements $\leq$ declared length
Insufficient elements	Missing elements auto-filled with null
Too many elements	Error
Auto-deduction	[] means length determined by init data
Mixed types	Same array can store different types

4.2 Variable Assignment

```
varName = expression;
```

```
arrayName [ indexExpr ] = expression;    !! Array element assignment
```

4.3 Variable Naming Rules

- (1) Starts with a letter or underscore
- (2) Followed by letters, digits, underscores
- (3) Cannot use keywords
- (4) Case-sensitive

4.4 Scope

- (1) Global scope: top-level variables
- (2) Function scope: variables defined inside functions
- (3) Block scope: variables defined in IF/WHILE blocks do not affect the outside

4.5 Global Variable Access

Inside a function, if a local variable is defined with the same name as a global variable, the global variable is shadowed.

To access a shadowed global variable from within a function, use the CALL keyword to declare it.

Syntax format:

```
CALL varName;
```

After declaration, the global variable can be accessed using @varName.

Examples:

```
DEFINE count = 10;      !! Global variable

MAIN FUNCTION() {

    DEFINE count = 20;  !! Local variable

    CALL count;         !! Declare access to global count

    OUTPUT count;       !! Output 20 (local variable)

    OUTPUT @count;      !! Output 10 (global variable)

}
```

Constraints:

- (1) CALL can only be used inside function bodies
- (2) The CALL variable must already be defined in the global scope
- (3) Without CALL declaration, global variables cannot be accessed via @

5. Expressions

5.1 Literal Expressions

- (1) Numbers: 10, 3.14
- (2) Strings: "hello"
- (3) Booleans: true, false

5.2 Identifier Expressions

Referencing variables: `x`, `name`

5.3 Binary Expressions

Operator	Precedence	Associativity
<code>* / %</code>	Highest	Left-associative
<code>+ -</code>	Medium	Left-associative
<code>> < >= <=</code>	Low	Left-associative
<code>== !=</code>	Lowest	Left-associative

5.4 Unary Expressions

`-expression`: numeric negation

5.5 Parenthesized Expressions

`(expression)`

5.6 Call Expressions

`funcName(arg1, arg2, ...)`

5.7 Array Access Expressions

`arrayVar [ indexExpr ] [ indexExpr ] ...`

5.8 Array Literal Expressions

`{ elem1, elem2, elem3, ... }`

Examples:

`!! Define array`

`DEFINE a[3] = {10, 20, 30};`

`!! Indices: 0, 1, 2`

```
DEFINE c[1 TO 3] = {10, 20, 30};           !! Indices: 1, 2, 3

!! Access array

OUTPUT a[0];           !! Output 10

OUTPUT a[1];           !! Output 20

OUTPUT a[2];           !! Output 30

OUTPUT c[1];           !! Output 10

OUTPUT c[3];           !! Output 30

OUTPUT c[0];           !! Error: index 0 out of range [1, 3]

!! Multi-dimensional array

DEFINE b[2][3] = {1,2,3}, {4,5,6};

OUTPUT b[0][1];        !! Output 2

OUTPUT b[1][2];        !! Output 6
```

5.9 TYPE expression

Syntax format:

```
TYPE(expression);
```

Function:

Get andReturnthe type name of the expression.

Return Values:

Type	Return value
null	"null"
number	"number"
string	"string"
bool	"bool"
array	"array"

Notes:

(1) TYPE is an expression, not a function, and can be combined with other expressions.

(2) TYPE can only be used to query types, not for type conversion

Examples:

```
DEFINE x = 10;
```

```
TYPE(x);      !! Return:  number
```

6. Statements

Type	Functional Statements	Non-functional Statements
Semantic Role	Perform runtime operations like computation, I/O, control flow	Declare identifiers, define entities, establish scope
Effective Phase	Runtime	Compile-time / Parse-time
Allowed at Top Level	Prohibited	Allowed
Typical Syntax	OUTPUT、IF、WHILE	DEFINE、FUNCTION、MAIN

6.1 Program Structure

A program consists of a sequence of statements, each ending with a semicolon ;.

Functional code must be written inside functions.

6.2 Block Statement

```
{
    stmt1;
    stmt2;
}
```

6.3 Variable Definition Statement

Syntax format:

```
DEFINE varName;                !! Define variable, initial value is null

DEFINE varName = expression;    !! Define variable and assign

DEFINE CONST constName = expression;    !! Define constant (must initialize)
```

Constraints:

- (1) Constants must be initialized at definition, cannot be declared separately
- (2) Once defined, a constant's value cannot be changed
- (3) Within the same scope constant names and variable names cannot conflict
- (4) Arrays can also be defined as constants: `DEFINE CONST arrayName dimDecl = arrayInit`

Examples:

```
DEFINE x;                !! Variable, value is null

DEFINE x = 10;           !! Variable, value is 10

x = 20;                  !! Can be modified

DEFINE CONST PI = 3.14;   !! Constant, value is 3.14

PI = 3.14159;            !! Error: constant cannot be reassigned

DEFINE CONST A;          !! Error: constant must be initialized

DEFINE CONST ARR[3] = {1, 2, 3};    !! Constant array

ARR[0] = 10;             !! Error: constant array elements cannot be modified
```

6.4 Assignment Statement

```
x = 20;
```

6.5 Output Statement

```
OUTPUT expression;
```

Output the value of the expression to standard output (no auto newline).

6.6 Input Statement

```
INPUT "prompt text" INTO variable;
```

```
INPUT INTO variable;    !! No prompt
```

Read a line of string from standard input into the variable.

6.7 Conditional Statement

```
IF (condition) {  
    statement;  
}
```

```
IF (condition) {  
    statement;  
} ELSE {  
    statement;  
}
```

Condition must be of boolean type.

6.8 Loop Statement

```
WHILE (condition) {  
    statement;  
}
```

Condition must be of boolean type.

6.9 WAIT Statement

Syntax format:

```
WAIT expression;
```

Wait for the specified number of seconds; the expression must be of numeric type. WAIT

blocks the current execution thread.

Examples:

```
WAIT 5;      !! Wait 5 seconds
```

```
WAIT 0.5;    !! Wait 0.5 seconds
```

```
DEFINE delay = 3;
```

```
WAIT delay;  !! Wait 3 seconds
```

6.10 Function Definition Statement

FunctionsIn BPSis a container

The purpose of a function definition is:

- (1) Give a block of code a name
- (2) Execute the code block when called

```
FUNCTION funcName(param1, param2) {  
    statement;  
    RETURN expression;  
}
```

Parameters are optional

RETURN statement is optional; returns null when no value is returned

6.11 Return Statement

```
RETURN expression;
```

Can only be used inside function bodies.

6.12 Main Function(Program entry point; must have exactly one.)

```
MAIN FUNCTION() {  
    statement;  
}
```

6.13 HTTP Request Statements

Syntax Format

- (1) GET "URL" SAVE variable;
- (2) POST "URL" WITH data SAVE variable;
- (3) PUT "URL" WITH data SAVE variable;
- (4) PATCH "URL" WITH data SAVE variable;
- (5) DELETE "URL" SAVE variable;

Parameter Description

Parameter	Description	Applicable Methods
"URL"	Request URL, string literal or expression	All methods
WITH data	Request data to carry (JSON string or form data)	POST、PUT、PATCH
SAVE variable	Save response result to variable	All methods

Example Code

```
MAIN FUNCTION()
{
    DEFINE response;

    DEFINE userId = 123;

    !! 1. GET request

    GET "https://httpbin.org/get?id=" + userId SAVE response;

    OUTPUT "GET Result: " + response + "\nL";

    !! 2. POST request (with data)

    POST "https://httpbin.org/post" WITH "{\"name\": \"ZhangSan\", \"age\": 25}"
```

SAVE response;

```
    OUTPUT "POST Result: " + response + "\nL";
```

```
!! 3. PUT request (with data)
```

```
    PUT "https://httpbin.org/put" WITH "{\id\:1,\status\:\updated\}" SAVE
response;
```

```
    OUTPUT "PUT Result: " + response + "\nL";
```

```
!! 4. PATCH request (with data)
```

```
    PATCH "https://httpbin.org/patch" WITH "{\age\:26}" SAVE response;
```

```
    OUTPUT "PATCH Result: " + response + "\nL";
```

```
!! 5. DELETE request
```

```
    DELETE "https://httpbin.org/delete?id=1" SAVE response;
```

```
    OUTPUT "DELETE Result: " + response + "\nL";
```

```
}
```

Note: WITH keyword can only be used with POST, PUT, PATCH

Data is typically in JSON string format

SAVE keyword saves the response; variable must be DEFINE'd first

6.14 Server Creation Statement

```
CREATESERVER port {
```

```
    ROUTE "path" {
```

```
        RESPONSE expression;
```

```
    }
```

```
    ROUTE "path" {
```

```

        FILE expression;

    }

    STATIC "directory";

}

```

Server runs in the background without blocking the main program

RESPONSE: return string response

FILE: return file content

STATIC: static file directory

6.15 Array Definition Statement

Syntax Format

```
DEFINE varName dimDecl = arrayInit;
```

Parameter Description

Parameter	Description
Variable name	Array name
Dimension declaration	One or more [N] or [N TO M] or []
Array initialization	{elem1, elem2, ...} or multiple {} combinations

6.16 Array Access Statement

Read element

```
arrayName [ indexExpr ];
```

Modify element

```
arrayName [ indexExpr ] = expression;
```

Example

```
DEFINE a[3] = {10, 20, 30};
```

```
OUTPUT a[0];           !! Read: output 10
```

```
a[0] = 100;           !! Modify  
  
OUTPUT a[0];         !! Output 100
```

```
DEFINE b[2][3] = {1,2,3}, {4,5,6};  
  
OUTPUT b[0][1];      !! Output 2  
  
b[1][2] = 100;       !! Modify  
  
OUTPUT b[1][2];      !! Output 100
```

```
!! Custom starting index  
  
DEFINE c[1 TO 3] = {10, 20, 30};  
  
OUTPUT c[1];         !! Output 10  
  
c[3] = 100;          !! Modify  
  
OUTPUT c[3];         !! Output 100
```

7. Built-in Functions

7.1 No Built-in Functions

The BPS-26 standard provides no built-in functions. All functionality is implemented through keyword statements.

8. Program Execution

8.1 Execution Entry Point

- (1) Parse all FUNCTION definitions
- (2) Locate MAIN FUNCTION
- (3) Execute statements in MAIN FUNCTION
- (4) Wait for all CREATESERVER servers to exit

8.2 Execution Method

bps.exe scriptfile.bps

9. Error Handling

9.1 Error Types

Type	Description
Syntax Error	Lexical/syntax error
Runtime Error	Runtime error (division by zero, undefined variable, etc.)
Type Error	Type mismatch

9.2 Error Report Format:

BPS <ErrorType>

File: <filename>

Line: <lineNumber>

Column: <columnNumber>

<sourceCodeLine>

^

<errorDescription>

Hint: <fixSuggestion>

10. Implementation Limitations

10.1 Known Limitations

- (1) No support for object/dictionary types
- (2) No support for standard library imports
- (3) Maximum string length: no hard limit (constrained by memory)

10.2 Unimplemented Features

None

Appendix A: Complete Syntax Examples

!! Variable definitions

```
DEFINE name = "BPS-26";
```

```
DEFINE version = 1.0;
```

```
DEFINE const = 10;
```

!! Constant definitions

```
DEFINE CONST PI = 3.14159;
```

```
DEFINE CONST APP_NAME = "BPS";
```

!! Array definitions

```
DEFINE arr[3] = {1, 2, 3};
```

```
DEFINE matrix[2][3] = {1,2,3}, {4,5,6};
```

```
DEFINE custom[1 TO 3] = {10, 20, 30};
```

```
DEFINE auto[] = {1, 2, 3, 4, 5};
```

!! Function definitions

```
FUNCTION add(a, b) {  
    RETURN a + b;  
}
```

```
!! Main function
```

```
MAIN FUNCTION() {
```

```
    !! Array access
```

```
    OUTPUT arr[0];           !! Output 1
```

```
    arr[1] = 100;           !! Modify element
```

```
    OUTPUT arr[1];           !! Output 100
```

```
    OUTPUT matrix[0][1];     !! Output 2
```

```
    matrix[1][2] = 99;       !! Modify element
```

```
    OUTPUT matrix[1][2];     !! Output 99
```

```
    OUTPUT custom[1];         !! Output 10 (starts from 1)
```

```
    OUTPUT custom[3];         !! Output 30
```

```
    OUTPUT auto[0];           !! Output 1 (auto-deduced length)
```

```
    OUTPUT auto[4];           !! Output 5
```

```
    !! HTTP request
```

```
    GET "https://api.example.com/data" SAVE response;
```

```
    OUTPUT response;
```

```
    !! HTTP server
```

```
    CREATESERVER 8080 {
```

```
    ROUTE "/" {  
        RESPONSE "Hello " + name;  
    }  
  
    ROUTE "/data" {  
        FILE "data.json";  
    }  
  
    STATIC "./static";  
}  
  
OUTPUT "BPS-26 Standard Started\\NL";  
  
DEFINE result = add(10, 20);  
  
OUTPUT "10+20=" + result + "\\NL";  
  
DEFINE count = 20;  
  
CALL count;  
  
OUTPUT @count;  
  
OUTPUT PI+"\\NL"+APP_NAME;  
  
WAIT 2; !! Wait 2 seconds  
  
OUTPUT "Program Ended\\NL";  
}
```

Appendix B: Standards Conformance Statement

This implementation (Basic Programming Script 0.0.2.3) fully conforms to all provisions of BPS-26.2 standard, with no additional extensions and no unimplemented features.

Appendix C: Token Number and Symbol Reference Table

When the parser reports an error showing expected token type X, consult the table below to find the Token type corresponding to that number for quick syntax troubleshooting.

BPS Programming Language Official Website:

bps.shdiv.net

Latest standards, version updates, and download resources, please visit the official website.

End of document.

Shanghai Advanced Digital Technologies reserves all rights to interpret and modify this document

Copyright © 2025-2026 Shanghai Advanced Digital Technologies Studio. All rights reserved.